

```
25 --
26
27 UPDATE reservations SET end_time = (CURRENT_TIMESTAMP - INTERVAL '1 hour') WHERE res_id = 2;
28 UPDATE spots SET current_status = 'Occupied' WHERE spot_id = (SELECT spot_id FROM reservations WHERE res_id
29
30
31
32
```

Data Output Messages Notifications

UPDATE 1

Query returned successfully in 77 msec.

Query Query History AI Assistant

```
3 spot_id INT REFERENCES spots(spot_id),
4 entry_time TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
5 exit_time TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
6 CONSTRAINT check_times CHECK (exit_time >= entry_time)
7 );
8
9 -- find a currently available spot
10 SELECT spot_id, current_status FROM spots WHERE spot_id = 2;
11
12 -- insert something in that spot
13 INSERT INTO sensor_events (spot_id, entry_time, exit_time)
14 VALUES (1, CURRENT_TIMESTAMP, NULL);
15 -- the 1 is for spot_id, like the one above
16
17 -- we print it again
18 SELECT spot_id, current_status FROM spots WHERE spot_id = 2;
19
20
```

Data Output Messages Notifications

spot_id	current_status
[PK] integer	character varying (20)
1	2 Available

Query Query History AI Assistant

```
1 -- find a currently available spot
2 SELECT spot_id, current_status FROM spots WHERE spot_id = 2;
3
4
5
```

Data Output Messages Notifications

spot_id	current_status
[PK] integer	character varying (20)
1	2 Available

```
49 END;
50 $$ LANGUAGE plpgsql;
51 -- then calling the function
52 CALL generate_tickets();
53 SELECT * FROM tickets ORDER BY ticket_id DESC LIMIT 2;
54
55
```

Data Output Messages Notifications

CALL

Query returned successfully in 60 msec.

```
52 CALL generate_tickets();
53 SELECT * FROM tickets ORDER BY ticket_id DESC LIMIT 2;
54
55
```

Data Output Messages Notifications

ticket_id	license_plate	violation_reason	amount	is_paid
[PK] integer	character varying (20)	character varying (100)	numeric (10,2)	boolean
1	14 VWX-234	Overtime Parking	50.00	false
2	13 MNO-345	Overtime Parking	50.00	false

In the images here are some explanations:

```
CREATE TABLE sensor_events (  
  event_id SERIAL PRIMARY KEY,  
  spot_id INT REFERENCES spots(spot_id),  
  entry_time TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  exit_time TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  CONSTRAINT check_times CHECK (exit_time >= entry_time)  
);
```

➔ We first add the table

```
SELECT spot_id, current_status FROM spots WHERE spot_id = 1;
```

➔ find a currently available spot

```
INSERT INTO sensor_events (spot_id, entry_time, exit_time)  
VALUES (2, CURRENT_TIMESTAMP, NULL);
```

➔ the 2 is for spot_id, like the one above

➔ insert something in that spot

```
SELECT spot_id, current_status FROM spots WHERE spot_id = 2;
```

➔ we print it again

```
SELECT COUNT(*) FROM tickets;
```

➔ we check the number of tickets we're at now

➔ We show the current # of tickets in the DB

```
CREATE OR REPLACE PROCEDURE generate_tickets()
```

```
AS $$
```

```
BEGIN
```

```

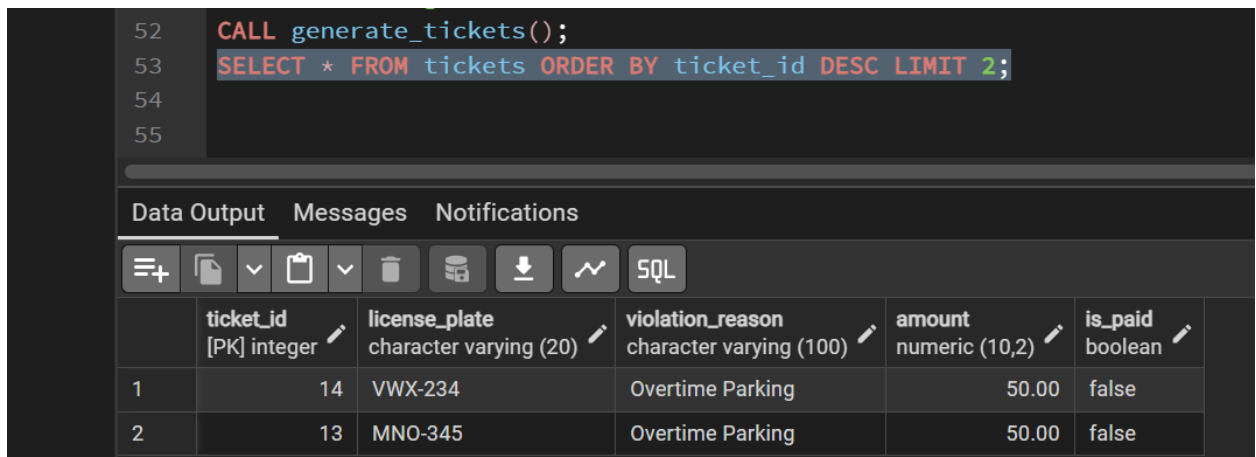
INSERT INTO tickets (license_plate, violation_reason, amount, is_paid)
SELECT v.license_plate, 'Overtime Parking', 50.00, FALSE
FROM reservations r
JOIN vehicles v ON r.user_id = v.owner_id
JOIN spots s ON r.spot_id = s.spot_id
WHERE r.end_time < CURRENT_TIMESTAMP -- Reservation is over
AND s.current_status = 'Occupied' -- Car hasn't left
-- Ensure we don't duplicate the same ticket
AND NOT EXISTS (
    SELECT 1 FROM tickets t
    WHERE t.license_plate = v.license_plate
    AND t.violation_reason = 'Overtime Parking'
    AND t.is_paid = FALSE
);
END;
$$ LANGUAGE plpgsql;
    ➔ then calling the function

```

```

CALL generate_tickets();
SELECT * FROM tickets ORDER BY ticket_id DESC LIMIT 2;
    ➔ Then when we run we get the final image, this

```



The screenshot shows a SQL IDE interface. At the top, a code editor displays the following SQL code:

```

52 CALL generate_tickets();
53 SELECT * FROM tickets ORDER BY ticket_id DESC LIMIT 2;
54
55

```

Below the code editor, there is a tabbed interface with 'Data Output', 'Messages', and 'Notifications'. The 'Data Output' tab is active, showing a table with the results of the SQL query. The table has the following columns and data:

	ticket_id [PK] integer	license_plate character varying (20)	violation_reason character varying (100)	amount numeric (10,2)	is_paid boolean
1	14	VWX-234	Overtime Parking	50.00	false
2	13	MNO-345	Overtime Parking	50.00	false